Graphic Era deemed to be University DEHRADUN

PROJECT-BASED LEARNING

PHASE-II EVALUATION

MiniGit: A Lightweight Version Control System Using C++ and Data Structures

Team ID: DSCPP-III-2026-T296

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun

Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T296
Semester: 3rd
Section: ML-1, ML-2, ML-3, ML-4
Domain: CSE (AI&ML)

Team Members

1. Gangwar, Abhinesh – 2510370192 (Team Lead)
2. Jain, Sparsh – 2510370159
3. Bajaj, Krish – 2510370469
4. Joshi, Snehil – 2510370146

Mentor

Mr. Himanshu Mainwal

Interactions completed: _____

Phase-I Recap & Feedback Incorporated

Phase-I Status

Local C++ CLI for repository creation, file tracking, commits, history, and recovery.

Feedback Received		Changes Incorporated
Architecture diagram needed correction	→	Architecture redrawn as five layers
Problem-solving approach needed improvement	→	Stack replaced by deque; singly linked list made doubly linked
Team self-review: justify data-structure choices	→	Dual-Trie ignore engine added; hash table replaced by `std::set`

Problem Statement & Finalized Objectives

Problem Statement

MiniGit is a lightweight local command-line version-control system in C++ for creating repositories, tracking files, saving commits, viewing history, and restoring earlier work.

Finalized Objectives

1. Create a local repository and manage file staging with `init`, `add`, and `remove`
2. Apply `.minigitignore` rules to files, directories, and extensions using a Dual Trie
3. Save commits with SHA-1 IDs, metadata, parent links, and snapshots
4. View commit history and manage saved states with `log`, `undo`, and `redo`

Updated System Architecture / Workflow



Major Components

- Handler routes CLI commands
- Repo/staging manages index, commits, undo
- Storage saves `.minigit` metadata/snapshots

Changes Since Phase-I

- Five layers: CLI, handler, repository, ignore, storage
- Dual-Trie ignore engine added
- Deque undo/redo; doubly linked commit history

Module Design & Functional Requirements

Major Modules

1. CLI / dispatcher
2. Dual-Trie ignore engine
3. Staging index (`std::set`)
4. CommitLog, undo/redo deque

Functional Requirements

- Initialize `.minigit` repository
- Stage/remove files; prevent duplicates
- Ignore paths/extensions
- Commit, undo/redo, view log and `log --reverse`

Module Dependency

Handler dispatches; only CLI prints results.

Technology Stack & Development Methodology

Technology Stack

Programming:	C++
Storage:	Flat files; no external DB
Libraries:	C++ Standard Library; C++14/C++17 bundled filesystem fallback
First-run install:	type `minigit` from any terminal
Development Tool:	VS Code
Version Control:	Git / GitHub

Development Methodology

1. Requirement Analysis
2. System Design
3. Module Development
4. Integration
5. Testing & Validation

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C++	Dual Trie; `std::set` staging index; deque undo/redo; doubly linked commit list	Ignore, staging, undo/redo, and CommitLog
OOPs with C++	Modular C++ components and result objects: `AddResult`, `IgnoreResult`, `CommitResult`	Command Handler and feature modules
Operating Systems	File/directory operations; `.minigit` metadata and commit snapshots	Storage layer
DBMS	No external database; flat-file HEAD, index, history, and commit objects	Storage layer

Development & Implementation Progress

Module	Status	Work Completed
Repository setup / CLI	Completed	`init`, `.minigit` layout, CLI parser/dispatcher, first-run install
Ignore engine and staging	Completed	Dual Trie, `.minigitignore`, add/remove, staging, and tests
Commit and state history	Completed	SHA-1 IDs, metadata, snapshots, parent links, deque undo/redo, tests
CommitLog and history	Completed	Doubly linked list, history loading, log / `log --reverse`, tests
Checkout / restore	Pending	Restore a chosen commit's files (planned for Phase-III)
Rollback	Pending	Undo the last restore (planned for Phase-III)

Overall Project Completion: about 67% (4 of 6 core commands)

Working Prototype / Demonstration

Implemented Features

- `init` creates the local repository and `.minigit` layout
- `add`/`remove` manage staged files with duplicate prevention
- Dual-Trie ignore engine reads `.minigitignore` rules
- `commit`, `undo`, `redo`, and `log` manage saved versions and history

Prototype / Output

```
usage: minigit <command> [<args>]

These are common MiniGit commands:
   init                Create an empty MiniGit repository or reinitialize an existing one
   add <file|dir>      Add file contents to the staging area index
   add .               Add all tracked/unignored repository files to index
   add -v <file>       Verify if file is staged in index
   remove <file|dir|.> Remove files from the staging area index
   ignore <pattern>    Add a pattern to .minigitignore
   ignore -r <pattern> Remove a pattern from .minigitignore
   ignore -v <path>    Verify whether a path is ignored and show line number
   install             Install MiniGit globally into system PATH
   uninstall           Uninstall MiniGit from system PATH
PS A:\Desktop\MiniGit> minigit init
Reinitialized existing MiniGit repository in A:\Desktop\MiniGit\.minigit
PS A:\Desktop\MiniGit> minigit add .
staged 73 file(s), 0 already staged, skipped 0 ignored
PS A:\Desktop\MiniGit>
```

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Trie unit test (`test_trie`)	Trie insert/search/remove; rule matching	Pass; `build/` != `buildings/`	Pass
Add/ignore unit test (`test_add`)	Staging, duplicates, rule lines	Pass; CRLF and rule lines checked	Pass
Commit and log unit tests	IDs, metadata, undo/redo, log order	Pass; 18 commit checks, log order	Pass

Results / Validation

Trie, add/ignore, commit, and log tests passed. Manual CLI passed; bad input was rejected.

GitHub Repository & Development Activity

Repository Information

- GitHub Link: strange-exe/MiniGit
- README.md: Updated
- Contributors: All 4 members added
- Repository Status: Public; 32 commits
- Branches: `main` + 5 member/UI branches

Development Activity

- 32 commits; latest: PR #7 merged
- Descriptive commit messages
- Individual contribution visible
- Latest work merged on `main`
- README and docs site maintained

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Abhinesh Gangwar	Team Lead	Repo setup, CLI, install, docs	20
Sparsh Jain	Developer	Dual Trie, ignore/staging tests	10
Krish Bajaj	Developer	SHA-1 commits, undo/redo	1
Snehil Joshi	Developer/Docs	CommitLog, log and `log --reverse` tests	1

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- Windows path mismatches
- Trie extension matching issue
- Separate-branch integration

Solutions / Learning

- `normalizePath()` standardizes paths
- Reverse Trie; shared module formats

Phase-III Roadmap

1. Checkout (Snehil Joshi, Krish Bajaj); rollback (Krish Bajaj)
2. Add checkout/rollback tests; improve storage
3. Sample repo, demo, README (Sparsh Jain)
4. Final report and live demo (all members)
5. Future scope: branching, merging, `log --graph`

Expected Outcomes & References

Expected Final Outcomes

- Working prototype: C++ CLI, from init to checkout and rollback
- Measurable results: unit tests pass; each command run on a sample repo
- Practical use: a small version-control tool students can read and run
- Deployment: one Windows executable, no dependencies
- Publication: unlikely; a teaching case study

References

1. E. Fredkin, "Trie Memory," *Commun. ACM*, vol. 3, no. 9, pp. 490–499, 1960.
2. D. Eastlake 3rd and P. Jones, *RFC 3174: US Secure Hash Algorithm 1 (SHA1)*, 2001.
3. S. Chacon and B. Straub, *Pro Git*, 2nd ed.
4. P. Morin, *Open Data Structures (in C++)* – opendatastructures.org/ods-cpp
5. cppreference.com, C++ Standard Library
6. Git Documentation – git-scm.com/doc
7. MiniGit Documentation Site

Thank You

Questions & Suggestions